EAST WEST UNIVERSITY

Course Code: CSE423

Course Title: Software Architecture

Section: 01

Semester: Spring26

Group No: 01

Report

Project Name: Hospital Management System

Submitted To:

Ahmed Adnan

Lecturer

Department of Computer Science and Engineering

Submitted By:

Name:	ID:
Sababa Hoque Saba	2022-3-60-060
Afia Ayub Samirah	2022-3-60-012
Kha. Mo. Syeed Asif	2022-3-60-030
Md. Omor Faruq Shikdere	2022-1-60-335

Date of Submission: 10-02-2026

Table of Content

Table of Content.....2
SOLID Principles Applied to Hospital Management System - AddDoctor Module.....3
1. Single Responsibility Principle (SRP).....3
2. Open/Closed Principle (OCP).....23
3. Liskov Substitution Principle (LSP).....24
4. Interface Segregation Principle (ISP).....24
5. Dependency Inversion Principle (DIP).....25

SOLID Principles Applied to Hospital Management System - AddDoctor Module

Link of Commit ([Applied SOLID Principle])

1. Single Responsibility Principle (SRP)

Before: **AddDoctor** class handled **UI layout, event handling, and doctor data storage**.

```
import java.lang.*;
```

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;



public class AddDoctor extends JFrame implements
MouseListener,ActionListener{



    JLabel userLabel,passLabel,genderLabel,programLabel,frame,imgLabel;

    JTextField userTF;

    JPasswordField passTF;

    JButton add, back;

    ButtonGroup bg1;

    JRadioButton r1,r2,r3;

    JComboBox dept;

    Color color1,color2;

    Font font1, font2, font3, font4;

    ImageIcon img;

    JPanel panel;
```

```java
public AddDoctor(){

    super("Doctor Information");
    this.setSize(900,600);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    this.setLocationRelativeTo(null);



    color1= new Color(143,207,225);
    color2= new Color(39,83,23);

    font1 = new Font("Biome",Font.BOLD, 30);
    font2 = new Font("Cascadia Code SemiBold",Font.PLAIN, 20);
    font3 = new Font("Cascadia Mono SemiBold",Font.PLAIN, 15);
    font4 = new Font("Biome",Font.BOLD, 18);



    panel = new JPanel();
    panel.setLayout(null);

    userLabel = new JLabel("HealthMate ");
    userLabel.setBounds(300,10,400,100);
```

```
        userLabel.setFont(font1);

        panel.add(userLabel);




        userLabel = new JLabel("Add Doctor Form ");

        userLabel.setBounds(130,80,400,50);

        userLabel.setFont(font4);

        panel.add(userLabel);


        userLabel = new JLabel("Doctor Name: ");

        userLabel.setBounds(75,120,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,160,150,20);

        panel.add(userTF);


        userLabel = new JLabel("Mobile Number: ");

        userLabel.setBounds(75,150,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,190,150,20);
```

```
        panel.add(userTF);


        genderLabel = new JLabel("Gender:");

        genderLabel.setBounds(75,220,115,20);

        panel.add(genderLabel);


        r1 = new JRadioButton("Male");

        r1.setBounds(150,220,80,20);

        r1.setBackground(color1);

        panel.add(r1);


        r2 = new JRadioButton("Female");

        r2.setBounds(230,220,80,20);

        r2.setBackground(color1);

        panel.add(r2);


        r3 = new JRadioButton("Other");

        r3.setBounds(320,220,80,20);

        r3.setBackground(color1);

        panel.add(r3);


        bg1 = new ButtonGroup();

        bg1.add(r1);
```

```
        bg1.add(r2);

        bg1.add(r3);




        programLabel = new JLabel("Department:");

        programLabel.setBounds(75,210,100,100);

        panel.add(programLabel);


        String items[] = {"CARDIOLOGY", "NEUROSURGERY", "ONCOLOGIST",
"ORTHOPEDICS"};

        dept = new JComboBox(items);

        dept.setBounds(170,250,150,20);

        panel.add(dept);


        userLabel = new JLabel("Joining Date: ");

        userLabel.setBounds(75,240,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,280,150,20);

        panel.add(userTF);


        userLabel = new JLabel("Salary: ");

        userLabel.setBounds(75,270,100,100);
```

```
        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,310,150,20);

        panel.add(userTF);


        userLabel = new JLabel("Room NO: ");

        userLabel.setBounds(75,300,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,340,150,20);

        panel.add(userTF);


        userLabel = new JLabel("BM&DC REG: ");

        userLabel.setBounds(75,330,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,370,150,20);

        panel.add(userTF);
```

```
add = new JButton("ADD");

add.setBounds(500,470,100,40);

add.setBackground(Color.GREEN);

add.setForeground(Color.WHITE);

add.setOpaque(true);

add.addMouseListener(this);

add.addActionListener(this);

panel.add(add);


back = new JButton("BACK");

back.setBounds(620,470,100,40);

back.setBackground(Color.RED);

back.setForeground(Color.WHITE);

back.setOpaque(true);

back.addMouseListener(this);

back.addActionListener(this);

panel.add(back);


img = new ImageIcon("images/emp-info.png");


imgLabel = new JLabel(img);

imgLabel.setBounds(485,100,300,300);
```

```
        panel.add(imgLabel);


        panel.setBackground(color1);

        this.add(panel);

    }


    public void mouseClicked(MouseEvent me){}

    public void mousePressed(MouseEvent me){}

    public void mouseReleased(MouseEvent me){}

    public void mouseEntered(MouseEvent me){

        if(me.getSource() == add){

            add.setBackground(Color.RED);

    add.setForeground(Color.WHITE);

        }else if(me.getSource() == back){

            back.setBackground(Color.GREEN);

            back.setForeground(Color.BLACK);

        }

    }

    public void mouseExited(MouseEvent me){

        if(me.getSource() == add){

            add.setBackground(Color.GREEN);

            add.setForeground(Color.BLACK);

        }else if(me.getSource() == back){
```

```
            back.setBackground(Color.RED);

            back.setForeground(Color.WHITE);

        }

    }



    public void actionPerformed(ActionEvent ae){

        String command = ae.getActionCommand();

        if(add.getText().equals(command));{

            DoctorList dl = new DoctorList();

            dl.setVisible(true);

            this.setVisible(false);



        }

        //public void actionPerformed(ActionEvent ae){

        //String command = ae.getActionCommand();

            if(ae.getSource() == back) {



                    JOptionPane.showMessageDialog(this, "Added doctor
successfully");

                     DrMenu dm = new DrMenu();

                    dm.setVisible(true);

                    this.setVisible(false);

                }
```

```
    }



}
```

After:

Doctor - handles **data only**

DoctorService - handles **doctor-related operations** (adding, storing)

AddDoctorUI - handles **UI only**

Doctor.java

```
public class Doctor {

    private String name;

    private String mobile;

    private String gender;

    private String department;

    private String joiningDate;

    private double salary;

    private String roomNo;

    private String bmdcReg;
```

```java
    public Doctor(String name, String mobile, String gender, String
department,
                  String joiningDate, double salary, String roomNo, String
bmdcReg) {
        this.name = name;
        this.mobile = mobile;
        this.gender = gender;
        this.department = department;
        this.joiningDate = joiningDate;
        this.salary = salary;
        this.roomNo = roomNo;
        this.bmdcReg = bmdcReg;
    }

    // Getters
    public String getName() { return name; }
    public String getMobile() { return mobile; }
    public String getGender() { return gender; }
    public String getDepartment() { return department; }
    public String getJoiningDate() { return joiningDate; }
    public double getSalary() { return salary; }
    public String getRoomNo() { return roomNo; }
    public String getBmdcReg() { return bmdcReg; }
}
```

DoctorService.java

```
import java.util.ArrayList;

import java.util.List;


public class DoctorService implements IDoctorService {

    private List<Doctor> doctors = new ArrayList<>();


    @Override

    public void addDoctor(Doctor doctor) {

        doctors.add(doctor);


    }


    public List<Doctor> getDoctors() {

        return doctors;

    }

}
```

AddDoctorUI.java

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;


public class AddDoctorUI extends JFrame {


    private IDoctorService doctorService;


    private JTextField nameTF, mobileTF, joiningTF, salaryTF, roomTF,
bmdcTF;

    private JRadioButton maleRB, femaleRB, otherRB;

    private JComboBox<String> deptCB;

    private JButton addButton, backButton;

    private ButtonGroup genderGroup;


    public AddDoctorUI(IDoctorService doctorService) {

        super("Add Doctor Form");

        this.doctorService = doctorService;


        setSize(900, 600);

        setLocationRelativeTo(null);
```

```
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);


        initUI();
    }


    private void initUI() {
        JPanel panel = new JPanel();
        panel.setLayout(null);
        panel.setBackground(new Color(143,207,225));


        Font labelFont = new Font("Cascadia Code SemiBold", Font.PLAIN,
18);


        // Labels & TextFields
        JLabel titleLabel = new JLabel("Add Doctor Form");
        titleLabel.setBounds(130, 20, 300, 40);
        titleLabel.setFont(new Font("Biome", Font.BOLD, 30));
        panel.add(titleLabel);


        JLabel nameLabel = new JLabel("Doctor Name:");
        nameLabel.setBounds(75, 80, 120, 25);
        nameLabel.setFont(labelFont);
        panel.add(nameLabel);
```

```
        nameTF = new JTextField();
        nameTF.setBounds(200, 80, 150, 25);
        panel.add(nameTF);

        JLabel mobileLabel = new JLabel("Mobile Number:");
        mobileLabel.setBounds(75, 120, 120, 25);
        mobileLabel.setFont(labelFont);
        panel.add(mobileLabel);

        mobileTF = new JTextField();
        mobileTF.setBounds(200, 120, 150, 25);
        panel.add(mobileTF);

        JLabel genderLabel = new JLabel("Gender:");
        genderLabel.setBounds(75, 160, 100, 25);
        genderLabel.setFont(labelFont);
        panel.add(genderLabel);

        maleRB = new JRadioButton("Male");
        maleRB.setBounds(150, 160, 80, 25);
        maleRB.setBackground(panel.getBackground());
        femaleRB = new JRadioButton("Female");
        femaleRB.setBounds(230, 160, 80, 25);
```

```
        femaleRB.setBackground(panel.getBackground());

        otherRB = new JRadioButton("Other");

        otherRB.setBounds(310, 160, 80, 25);

        otherRB.setBackground(panel.getBackground());


        genderGroup = new ButtonGroup();

        genderGroup.add(maleRB);

        genderGroup.add(femaleRB);

        genderGroup.add(otherRB);


        panel.add(maleRB);

        panel.add(femaleRB);

        panel.add(otherRB);


        JLabel deptLabel = new JLabel("Department:");

        deptLabel.setBounds(75, 200, 120, 25);

        deptLabel.setFont(labelFont);

        panel.add(deptLabel);


        String[] departments = {"CARDIOLOGY", "NEUROSURGERY",
"ONCOLOGIST", "ORTHOPEDICS"};

        deptCB = new JComboBox<>(departments);

        deptCB.setBounds(200, 200, 150, 25);

        panel.add(deptCB);
```

```
JLabel joiningLabel = new JLabel("Joining Date:");
joiningLabel.setBounds(75, 240, 120, 25);
joiningLabel.setFont(labelFont);
panel.add(joiningLabel);

joiningTF = new JTextField();
joiningTF.setBounds(200, 240, 150, 25);
panel.add(joiningTF);

JLabel salaryLabel = new JLabel("Salary:");
salaryLabel.setBounds(75, 280, 120, 25);
salaryLabel.setFont(labelFont);
panel.add(salaryLabel);

salaryTF = new JTextField();
salaryTF.setBounds(200, 280, 150, 25);
panel.add(salaryTF);

JLabel roomLabel = new JLabel("Room No:");
roomLabel.setBounds(75, 320, 120, 25);
roomLabel.setFont(labelFont);
panel.add(roomLabel);
```

```
roomTF = new JTextField();
roomTF.setBounds(200, 320, 150, 25);
panel.add(roomTF);

JLabel bmdcLabel = new JLabel("BM&DC REG:");
bmdcLabel.setBounds(75, 360, 120, 25);
bmdcLabel.setFont(labelFont);
panel.add(bmdcLabel);

bmdcTF = new JTextField();
bmdcTF.setBounds(200, 360, 150, 25);
panel.add(bmdcTF);

// Buttons
addButton = new JButton("ADD");
addButton.setBounds(500, 470, 100, 40);
addButton.setBackground(Color.GREEN);
addButton.setForeground(Color.WHITE);
addButton.addActionListener(e -> onAdd());
panel.add(addButton);

backButton = new JButton("BACK");
```

```
        backButton.setBounds(620, 470, 100, 40);

        backButton.setBackground(Color.RED);

        backButton.setForeground(Color.WHITE);

        backButton.addActionListener(e -> onBack());

        panel.add(backButton);



        this.add(panel);

    }



    private void onAdd() {

        String name = nameTF.getText();

        String mobile = mobileTF.getText();

        String gender = maleRB.isSelected() ? "Male" :

                        femaleRB.isSelected() ? "Female" : "Other";

        String department = (String) deptCB.getSelectedItem();

        String joiningDate = joiningTF.getText();

        double salary = Double.parseDouble(salaryTF.getText());

        String room = roomTF.getText();

        String bmdc = bmdcTF.getText();



        Doctor doctor = new Doctor(name, mobile, gender, department,
joiningDate, salary, room, bmdc);

        doctorService.addDoctor(doctor);
```

```
        JOptionPane.showMessageDialog(this, "Doctor added successfully!");
        clearFields();
    }

    private void onBack() {
        this.setVisible(false);
        DrMenu dm = new DrMenu(); // Navigation to main menu
        dm.setVisible(true);
    }

    private void clearFields() {
        nameTF.setText("");
        mobileTF.setText("");
        genderGroup.clearSelection();
        deptCB.setSelectedIndex(0);
        joiningTF.setText("");
        salaryTF.setText("");
        roomTF.setText("");
        bmdcTF.setText("");
    }
}
```

2. Open/Closed Principle (OCP)

Before: Adding new fields required **modifying AddDoctor class**.

Violation in Original Code

- To add a new role (Doctor, Nurse, Admin), existing menu code had to be modified.
- Logic was hardcoded inside UI classes.

Refactored Solution

- New features can be added by:

  Creating new service classes
  Implementing interfaces

After: Doctor class stores doctor attributes. Adding a new field can be done by extending Doctor or using a builder pattern **without modifying existing logic**.

```
public interface IDoctorService {

    void addDoctor(Doctor doctor);

}
```

3. Liskov Substitution Principle (LSP)

Violation in Original Code

- UI classes depended directly on concrete implementations.
- No abstraction existed.

Refactored Solution

UI depends on the interface, not the implementation.Any class implementing **IDoctorService** can replace **DoctorService**.

```
public interface IDoctorService {

    void addDoctor(Doctor doctor);

}
```

If logic changed, the UI would break.

4. Interface Segregation Principle (ISP)

UI depends on **IDoctorService** interface, not concrete implementations.

Classes implement only the methods they need.
UI uses **only what it needs**

No unnecessary method dependency

```
public interface IDoctorService {

    void addDoctor(Doctor doctor);
```

```
}
```

5. Dependency Inversion Principle (DIP)

Violation in Original Code

- **UI directly created service objects.**
- **Strong coupling between UI and logic.**

AddDoctorUI depends on **IDoctorService** abstraction, not **DoctorService** directly.

Allows swapping implementations or mocking for testing.

```
public class Start {
    public static void main(String[] args) {
        IDoctorService doctorService = new DoctorService();
        AddDoctorUI addDoctorUI = new AddDoctorUI(doctorService);
        addDoctorUI.setVisible(true);
    }
}
```